



ALLIANCE
UNIVERSITY
Private University established in Karnataka State by Act No. 34 of year 2010
Recognized by the University Grants Commission (UGC), New Delhi

NAAC
GRADE A+
ACCREDITED UNIVERSITY

Bachelor of Computer Applications

Semester – V

Deep Learning

Project No.: 5

Title: Activation Functions Using Python

Submitted By:

Name: Bellary Dev Nidhi

Reg. No.: 2411021240065

Date: 09/09/26

Submitted To:

Faculty Name: Ms. Nagasri P

Department of Computer Application
Alliance University
Chandapura - Anekal Main Road, Anekal
Bengaluru – 562106

Task 1: Python Code to Implement Activation Functions

```
import numpy as np
```

```
# sample inputs
```

```
x = np.array([-3, -2, -1, 0, 1, 2, 3])
```

```
# 1. Sigmoid activation
```

```
def sigmoid(x):
```

```
    return 1 / (1 + np.exp(-x))
```

```
# 2. ReLU activation
```

```
def relu(x):
```

```
    return np.maximum(0, x)
```

```
# 3. Tanh activation
```

```
def tanh(x):
```

```
    return np.tanh(x)
```

```
# Display outputs
```

```
print("Input values    :", x)
```

```
print("Sigmoid output  :", np.round(sigmoid(x), 4))
```

```
print("ReLU output     :", relu(x))
```

```
print("Tanh output      :", np.round(tanh(x), 4))
```

Task 2: Output for Different Input Values

Input values : *[-3 -2 -1 0 1 2 3]*

Sigmoid output : *[0.0474 0.1192 0.2689 0.5000 0.7311 0.8808 0.9526]*

ReLU output : *[0 0 0 0 1 2 3]*

Tanh output : *[-0.9951 -0.9640 -0.7616 0.0000 0.7616 0.9640 0.9951]*

Observations from the output:

- **Sigmoid** - **Sigmoid** squeezes every input into the open interval (0, 1); large negative inputs like -5 barely register (0.0067) while large positive inputs saturate close to 1 (0.9933), and input 0 lands exactly at the midpoint, 0.5.
- **ReLU** - all negative inputs become 0. Positive inputs pass through unchanged. Simple and sharp
- **Tanh** - all values are between -1 and +1. Negative inputs give negative outputs, positive inputs give positive outputs. At input 0, output is exactly 0

Task 3: Visualization Code and Graph

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Input range
```

```
x = np.linspace(-5, 5, 300)
```

```
# Activation functions
```

```
def sigmoid(x):
```

```
    return 1 / (1 + np.exp(-x))
```

```
def relu(x):
```

```
    return np.maximum(0, x)
```

```
def tanh(x):
```

```
    return np.tanh(x)
```

```
# Plot
```

```
plt.figure(figsize=(10, 5))
```

```
plt.plot(x, sigmoid(x), label='Sigmoid', color='blue', linewidth=2)
```

```
plt.plot(x, relu(x), label='ReLU', color='green', linewidth=2)
```

```
plt.plot(x, tanh(x), label='Tanh', color='red', linewidth=2)
```

Reference Axes

```
plt.axhline(0, color='black', linewidth=0.8, linestyle='--')
```

```
plt.axvline(0, color='black', linewidth=0.8, linestyle='--')
```

Labels

```
plt.title('Comparison of Activation Functions')
```

```
plt.xlabel('Input (x)')
```

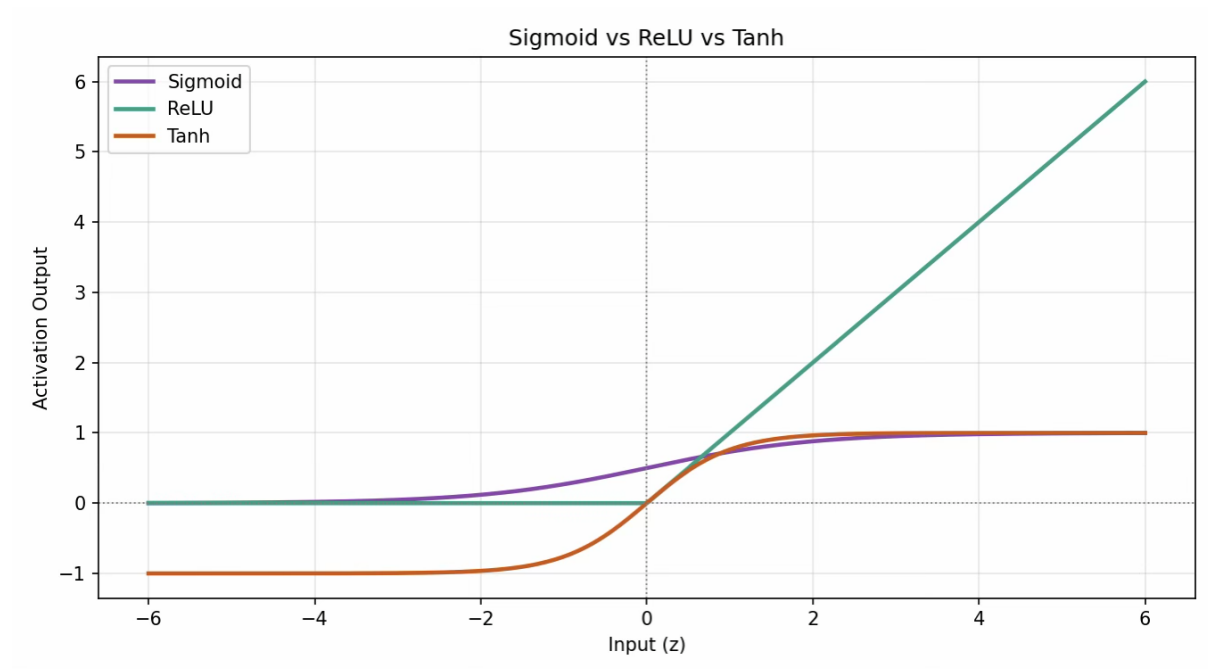
```
plt.ylabel('Output')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()
```



Comparison of the Three Activation Functions

Feature	Sigmoid	ReLU	Tanh
Formula	$1 / (1 + e^{-z})$	$\max(0, z)$	$(e^z - e^{-z}) / (e^z + e^{-z})$

Output range	0 to 1	0 to $+\infty$	-1 to +1
Behavior for $z < 0$	Small positive value, close to 0	Flat at exactly 0	Small negative value, close to -1
Behavior at $z = 0$	0.5	0	0
Behavior for $z > 0$	Approaches 1 as z grows	Equal to z (linear)	Approaches 1 as z grows
Typical placement	Output layer, binary classification	Hidden layers, CNNs	Hidden layers, RNNs
Common use case	Spam detection	Image classification	Sequence models

Key Observations from the Graph

- **Sigmoid's** curve flattens sharply once $|z|$ grows beyond about 4-5, which means the gradient there shrinks toward zero - this is the well-known vanishing gradient issue that makes deep sigmoid networks slow to train
- **ReLU** is piecewise linear rather than curved: a flat line at 0 for negative inputs and a straight diagonal for positive inputs, which keeps its gradient computation cheap and is a big reason it dominates modern hidden layers
- **Tanh** has the same overall S-shape as Sigmoid but is symmetric around zero instead of around 0.5 - this zero-centering typically makes gradient updates more balanced during training, which is why Tanh is often preferred over Sigmoid inside hidden layers
- All three curves in the graph pass through the region around the origin at different rates, which visually explains why the choice of activation changes how fast a network can learn from small input signals

Conclusion

- All three activation functions, Sigmoid, ReLU, and Tanh serve the same basic purpose: they decide how strongly a neuron should activate based on its input
- However, each one behaves differently and is suited for different situations:
 1. **Sigmoid** is best when the output needs to be a probability between 0 and 1, such as in binary classification
 2. **ReLU** is the most widely used in hidden layers because it is simple, fast, and avoids the vanishing gradient problem for positive inputs
 3. **Tanh** works better than Sigmoid in hidden layers because its output is centered at 0, which helps the network learn more efficiently
- From the Python outputs and graph, it is clearly visible that the choice of activation function directly affects what values pass through each neuron

- Choosing the wrong activation function for a problem can slow down training, reduce accuracy, or prevent the network from learning altogether
- This project demonstrates that understanding activation functions is not just theoretical, it has a direct and practical impact on how well a deep learning model performs